

Efficient state space generation of GSPNs using decision diagrams

Andrew S. Miner

Department of Computer Science, Iowa State University

E-mail: asminer@iastate.edu

Abstract

Implicit techniques for representing and generating the reachability set of a high-level model have become quite efficient. However, such techniques are usually restricted to models whose events have equal priority. Models containing events with differing classes of priority or complex priority structure, in particular models with immediate events, have thus been required to use explicit reachability set generation techniques. In this paper, we present an efficient implicit technique, based on multi-valued decision diagram representations for sets of states and matrix diagram representations for next-state functions, that can handle models with complex priority structure. If the model contains immediate events, the vanishing states can be eliminated either during generation, by manipulating the matrix diagram, or after generation, by manipulating the multi-valued decision diagram. We apply both techniques to several models and give detailed results.

1. Introduction

The generation of the state space, or reachability set, for a discrete-state model such as a stochastic Petri net (SPN), generalized stochastic Petri net (GSPN) or queueing network, is a necessary step in many types of studies. Formal verification techniques, such as model checking, may require generation of the reachability set to verify that the reachable states all satisfy some desirable property, such as the absence of a deadlock. Performance evaluation, based on Markov chain analysis, often uses the reachability set of a high-level model while constructing its underlying Markov chain and while computing reward measures once the Markov chain has been analyzed. In either case, generation of the reachability set is often difficult in practice due to the state explosion problem: the reachability set can be extremely large, even for a “small” model.

Traditional approaches to generate the reachability set $\mathcal{S}$ are *explicit*: reachable states are examined and generated one at a time, using an efficient data structure to store the

discovered states [10, 19]. Determination of the states that can be reached from the current state is done by examining the high-level model. This allows for maximum flexibility: the behavior of events can depend on the system state in complex ways.

Work has also been done on explicit techniques that exploit structured high-level representations, such as superposed GSPNs and stochastic automata networks. For these systems, the *next-state* function $\mathcal{N}$, which specifies the states that can be reached from the current state in a single step, can be represented in terms of Kronecker products and sums of several small matrices [7, 31]. Explicit data structures, based on the Kronecker representation, can be used to efficiently describe the reachability set [6, 23, 22]. Given current technology, explicit techniques are limited to about 10^8 states at best.

Recent advancements in reachability set generation based instead on *implicit* techniques have been quite promising. Unlike explicit techniques, implicit techniques do not necessarily require space or time proportional to the size of the reachability set. This allows implicit techniques to handle reachability set sizes of 10^{20} states or more. The use of *decision diagrams*, binary decision diagrams (BDDs) [5] in particular, has been embraced by many researchers following the work of Clarke et al. [8, 9]. Pastor et al. [29, 30] developed BDD-based techniques for generating the reachability set for *safe* Petri nets. The reachability set and the next-state function are represented implicitly using BDDs. The authors suggest that non-safe nets can be handled by encoding the integer number of tokens in a place using several binary variables, although this significantly complicates the next-state function.

Alternate approaches to handle non-safe Petri nets have been developed by Ciardo et al. [14, 15, 27]. These implicit approaches use Kronecker representations for the next-state function, and multi-valued decision diagrams (MDDs) [21, 32] to represent the reachability set, allowing state variables to be bounded integers. To use the Kronecker representation, a model must be partitioned so that each event’s contribution to the next-state function can be expressed as the cross-product of “local” functions. Separation of the

events according to their affected submodels allows for an efficient iteration strategy called *saturation* [15], which can dramatically reduce the time and memory required to generate the reachability set.

A substantial limitation of previous implicit techniques for reachability set generation for Petri nets is that all transitions are assumed to have equal priority. This implies that all transitions must be timed, since immediate transitions have priority over timed transitions. Immediate transitions can be handled only if they are completely *local* to a submodel: local immediate transitions are removed and the submodel is re-defined appropriately. The focus and contribution of this paper is an implicit technique that can handle transitions with complex priority structure, and thus can handle immediate transitions in general.

Once the reachability set has been generated, it can be used to verify properties of the system. If we are interested in performance evaluation, the reachability set can be used to generate the underlying Markov chain of the model. With current technology, Markov chains with perhaps 10^7 to 10^8 states can be analyzed exactly. This corresponds to the sizes of reachability sets that can be generated with explicit techniques. However, the ability to represent large reachability sets and Markov chains *exactly* using implicit techniques opens up intriguing possibilities for approximation techniques [28]. Thus, implicit reachability set generation algorithms are important for both model checking and performance evaluation.

The remainder of the paper is organized as follows. Additional background information about MDDs is given in Section 2. In Section 3 we propose an alternate representation of the next-state functions, based on *matrix diagrams* [18, 25, 26]. We then show in Section 4 how the next-state functions can be manipulated to handle events with different priorities, including immediate events. In Section 5 we describe two methods for eliminating vanishing states to obtain the tangible states only: elimination “on-the-fly” and elimination after generation. Experimental results, based on our implementation in SMART [16], are given in Section 6. Finally, Section 7 concludes the work.

2. Background and related work

The reachability set is formally defined as follows. Given a finite set of possible states $\hat{S}$, an initial state $s \in \hat{S}$ (or a set of initial states), and a next-state function $\mathcal{N} : \hat{S} \rightarrow 2^{\hat{S}}$ which describes the (possibly empty) set of states that may be reached from a given state in a single step, the reachability set $\mathcal{S}$ is the smallest set containing the initial state(s) that is closed over $\mathcal{N}$. A common technique to determine $\mathcal{S}$ is to compute the sequence $\mathcal{S}^n = \mathcal{S}^{n-1} \cup \mathcal{N}(\mathcal{S}^{n-1})$, where $\mathcal{S}^n$ is the set of states reachable from the initial state in n steps or fewer, with $\mathcal{S}^0 = \{s\}$. Since the set $\hat{S}$ is finite and

$\mathcal{S}^n \subseteq \mathcal{S}^{n+1} \subseteq \hat{S}$ for all $n \in \mathbb{N}$, the sequence will eventually converge with $\lim_{n \rightarrow \infty} \mathcal{S}^n = \mathcal{S}$. Note that for this paper, we consider finite reachability sets only.

The practical goal of any reachability set generation algorithm is to efficiently represent $\hat{S}$ and $\mathcal{N}$, while allowing for efficient computation of $\mathcal{S}$. In the case of SPNs, the set $\hat{S}$ can be implicitly specified by placing an upper bound on the number of tokens allowed in each place. Another approach is to decompose the model into K submodels, for some $K \in \mathbb{N}$. This is done for SPNs by partitioning the set of places $\mathcal{P}$. The finest possible decomposition is obtained when each submodel consists of a single place, $K = |\mathcal{P}|$; we can also group places together so that submodels contain multiple places. Given a decomposition, the set $\hat{S}$ can be written as $\hat{S} = \mathcal{S}_K \times \cdots \times \mathcal{S}_1$, where $\mathcal{S}_k$ is the set of possible substates for submodel k . A model state can thus be expressed as a K -tuple $(s_K, \dots, s_1)$ with $s_k \in \mathcal{S}_k$.

While the set $\mathcal{S}$ is assumed to be finite, it is often extremely large. One way to combat this problem is through the use of an efficient data structure. In particular, a set of states can be represented using a multi-valued decision diagram (MDD) [21, 32]. If the model is composed of K submodels, then an MDD representing a set of states is a directed, acyclic graph consisting of *terminal* nodes zero and one, and *non-terminal* nodes, each corresponding to a submodel. We say a non-terminal node is a *level- k* node if it corresponds to submodel k , and we say that terminal nodes are level-0 nodes. A level- k non-terminal node contains $|\mathcal{S}_k|$ downward pointers, one for each substate in $\mathcal{S}_k$. A state is contained in the set represented by the MDD if and only if the path from the root node, following the pointers corresponding to the appropriate substates, reaches terminal node one. We say an MDD is *ordered* if all pointers from a level- k node are either to a level- $(k-1)$ node or to terminal node zero, and is *quasi-reduced* if it does not contain two non-terminal nodes whose corresponding pointers are equal. For the remainder of the paper, we assume all MDDs are ordered and quasi-reduced. An example MDD and the set of states it encodes is shown in Figure 1.

For traditional reachability set generation, the function $\mathcal{N}$ is specified by the high-level model. Many types of formalisms, including SPNs, specify $\mathcal{N}$ in terms of distinct events:

$$\mathcal{N}(s) = \bigcup_{e \in \mathcal{E}} \mathcal{N}^e(s) \quad (1)$$

where $\mathcal{E}$ is a finite set of events. In the case of SPNs, events correspond to transitions. For implicit techniques, it is usually assumed that each $\mathcal{N}^e$ function can be expressed as the cross-product of local functions [14, 15, 25, 27]:

$$\mathcal{N}^e(s_K, \dots, s_1) = \mathcal{N}_K^e(s_K) \times \cdots \times \mathcal{N}_1^e(s_1) \quad (2)$$

where $\mathcal{N}_k^e : \mathcal{S}_k \rightarrow 2^{\mathcal{S}_k}$. If this property does not hold for some event, we can either re-partition the model so that it

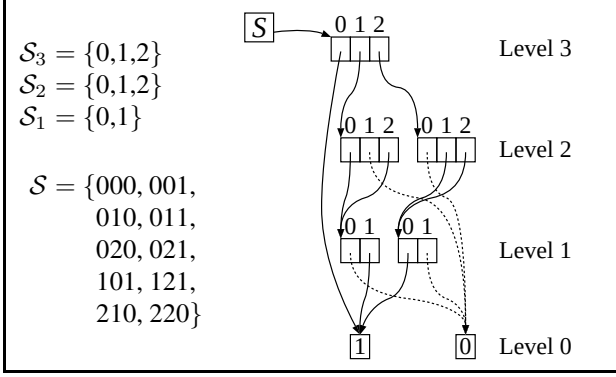


Figure 1. An MDD and its represented states

does hold, or split the event into sub-events such that the property holds for the sub-events. The function $\mathcal{N}$ can thus be represented by tables for each of the $\mathcal{N}_k^e$ functions. Note that this is equivalent to using a Kronecker representation for the next-state function [6, 22, 23].

3. Representing the next-state function with matrix diagrams

Matrix diagrams were initially introduced as an alternative to Kronecker representations for the transition rate matrix [18, 25]. In this paper, we describe a simplified version of the Canonical Matrix Diagrams presented in [26]: the matrices presented here can only have values zero or one, and are all the same size.

A *matrix diagram* is similar to an MDD, except the non-terminal nodes are matrices of pointers instead of vectors of pointers. Matrix diagrams consist of two level-0 terminal nodes, zero and one, and level- k non-terminal nodes, for $k \in \{1, \dots, K\}$. Each level- k non-terminal node contains a $|\mathcal{S}_k| \times |\mathcal{S}_k|$ matrix of pointers to other nodes. We say a matrix diagram is *ordered* if the pointers from a level- k node all point to nodes at levels less than k , and a matrix diagram is *quasi-reduced* if no two level- k nodes have equal matrices.

A matrix diagram represents a boolean matrix of size $\hat{\mathcal{S}} \times \hat{\mathcal{S}}$, which can encode a next-state function: the non-zero entries in a given row correspond to the states that can be reached from that row. We denote a next-state function in script, as $\mathcal{N}$, and its corresponding matrix diagram representation in boldface, as $\mathbf{N}$. In matrix diagram $\mathbf{N}$, state $(j_K, \dots, j_1)$ is contained in $\mathcal{N}(i_K, \dots, i_1)$ if the path from the root node, following pointer $[i_k, j_k]$ from level k , reaches terminal node one. An example (quasi-reduced, ordered) matrix diagram $\mathbf{N}$ and the next-state function $\mathcal{N}$ it represents are shown in Figure 2. For clarity, pointers to terminal node zero are omitted. Note that several states are

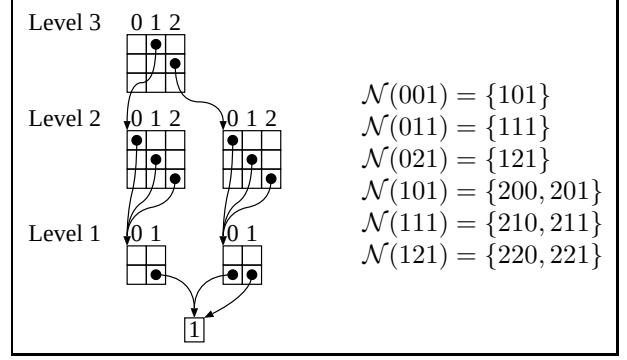


Figure 2. Representing a next-state function with a matrix diagram

not listed (for instance, $\mathcal{N}(000)$); the next-state function returns the empty set for the states that are not shown.

An important characteristic captured by matrix diagram structure is the independence of the next-state function from a given submodel. In this case, the matrix diagram nodes for that submodel will always be an “identity matrix” with equal pointers. That is, the pointers along the diagonal will be equal, and the off-diagonal pointers will be to terminal node zero. In the example in Figure 2, the next-state function does not depend on submodel 2. This behavior naturally occurs in the high-level model when an event is independent of a submodel (for instance, if a transition is not connected to any place within a subnet). To exploit this property, we can define another reduction rule. We say a matrix diagram is *reduced* if it is quasi-reduced and it does not contain a non-terminal node whose diagonal pointers are all to the same node and whose non-diagonal pointers are all to terminal node zero. For the remainder of the paper, we assume all matrix diagrams are ordered and reduced. For example, the matrix diagram shown in Figure 2 would not actually store the nodes for level 2; instead, the pointers from the level-3 node would jump directly to the level-1 nodes. Thus, terminal node zero corresponds to an appropriately-sized square matrix of all zeroes, and terminal node one corresponds to an appropriately-sized identity matrix.

3.1. Constructing next-state functions

If the next-state function is represented by a function for each event (as in Equation 1), and the event functions are expressible as the cross-product of local functions (as in Equation 2), then the algorithm to construct a matrix diagram $\mathbf{N}^e$ for the next-state function $\mathcal{N}^e$ is straightforward, and is detailed in Figure 3. Given the local functions $\mathcal{N}_K^e, \dots, \mathcal{N}_1^e$, we can construct a matrix diagram node at level k based on

```

BuildNext( $\mathcal{N}_K^e, \dots, \mathcal{N}_1^e$ )
1:  $\mathbf{N}^e \leftarrow \text{one}$ ;
2: for each  $k \in \{1, \dots, K\}$  do
3:   if  $\mathcal{N}_k^e$  is not identity then
4:      $\mathbf{C} \leftarrow$  New level- $k$  node of all zeroes;
5:     for each  $i \in \mathcal{S}_k, j \in \mathcal{N}_k^e(i)$  do
6:        $\mathbf{C}[i, j] \leftarrow \mathbf{N}^e$ ;
7:     end for
8:      $\mathbf{N}^e \leftarrow \mathbf{C}$ ;
9:   end if
10: end for
11: return  $\mathbf{N}^e$ ;

```

Figure 3. Constructing a next-state function for a given event

$\mathcal{N}_k^e$. If $\mathcal{N}_k^e$ is the identity function, then the matrix diagram $\mathbf{N}^e$ will skip level k . Otherwise, row i_k of the matrix diagram node at level k will contain a pointer in column j_k if $j_k \in \mathcal{N}_k^e(i_k)$. All pointers for a given node will be equal, and will point to the next non-identity node or terminal node one.

3.2. Manipulating matrix diagrams

The main strength of matrix diagrams over Kronecker-based representations is their ability for manipulation. Binary operations on MDDs and matrix diagrams have complexity that depends on the number of nodes in the graphs, rather than the number of items encoded by the graph. Thus, the compactness in size of MDDs and matrix diagrams leads not only to obvious memory savings, but also to computational savings.

Two important operations on matrix diagrams for this work are matrix addition and matrix multiplication. Given next-state functions $\mathcal{A}$ and $\mathcal{B}$, we can compute the union $\mathcal{C}$ of the two functions (i.e., $\mathcal{C}(x) = \mathcal{A}(x) \cup \mathcal{B}(x)$) by adding their matrix diagrams $\mathbf{A}$ and $\mathbf{B}$. Similarly, the composition $\mathcal{C}$ of functions $\mathcal{A}$ and $\mathcal{B}$ (i.e., $\mathcal{C}(x) = \mathcal{B}(\mathcal{A}(x))$) can be determined by multiplying matrix diagrams $\mathbf{A}$ and $\mathbf{B}$. The algorithm for matrix diagram addition is shown in Figure 4, where $\text{BinaryAdd}(\mathbf{A}, \mathbf{B})$ computes $\mathbf{A} + \mathbf{B}$ for boolean matrices. The algorithm for matrix diagram multiplication is shown in Figure 5, where $\text{BinaryMult}(\mathbf{A}, \mathbf{B})$ computes $\mathbf{A} \cdot \mathbf{B}$ for boolean matrices. Note that both algorithms are recursive. The recursion terminates if the appropriate terminal conditions hold, or if the answer has already been computed. For addition, the terminal conditions are $\mathbf{A} + \mathbf{A} = \mathbf{A} + \mathbf{0} = \mathbf{A}$ (line 1) and $\mathbf{0} + \mathbf{B} = \mathbf{B}$ (line 3). For multiplication, the terminal conditions are $\mathbf{A} \cdot \mathbf{0} = \mathbf{0} \cdot \mathbf{B} = \mathbf{0}$ (line 1), $\mathbf{I} \cdot \mathbf{B} = \mathbf{B}$ (line 3), and $\mathbf{A} \cdot \mathbf{I} = \mathbf{A}$ (line 5). Since matrix diagram pointers may be equal, it is possible that a

```

BinaryAdd(Md A, Md B)
1: if  $(\mathbf{A} = \mathbf{B}) \vee (\mathbf{B} = \text{zero})$  then
2:   return  $\mathbf{A}$ ;
3: else if  $(\mathbf{A} = \text{zero})$  then
4:   return  $\mathbf{B}$ ;
5: else if AddCache contains  $(\mathbf{A}, \mathbf{B}, \mathbf{C})$  then
6:   return  $\mathbf{C}$ ;
7: end if
8:  $k_A \leftarrow \text{Level}(\mathbf{A})$ ;  $k_B \leftarrow \text{Level}(\mathbf{B})$ ;
9:  $k \leftarrow \text{MAX}(k_A, k_B)$ ;
10:  $\mathbf{C} \leftarrow$  New level- $k$  node;
11: if  $k_A > k_B$  then
12:    $\forall i, \mathbf{C}[i, i] \leftarrow \text{BinaryAdd}(\mathbf{A}[i, i], \mathbf{B})$ ;
13:    $\forall i, j, i \neq j, \mathbf{C}[i, j] \leftarrow \mathbf{A}[i, j]$ ;
14: else if  $k_B > k_A$  then
15:    $\forall i, \mathbf{C}[i, i] \leftarrow \text{BinaryAdd}(\mathbf{A}, \mathbf{B}[i, i])$ ;
16:    $\forall i, j, i \neq j, \mathbf{C}[i, j] \leftarrow \mathbf{B}[i, j]$ ;
17: else
18:    $\forall i, j, \mathbf{C}[i, j] \leftarrow \text{BinaryAdd}(\mathbf{A}[i, j], \mathbf{B}[i, j])$ ;
19: end if
20: Reduce  $\mathbf{C}$ ;
21: Insert AddCache entry  $(\mathbf{A}, \mathbf{B}, \mathbf{C})$ ;
22: return  $\mathbf{C}$ ;

```

Figure 4. Matrix diagram addition

single call to BinaryAdd or BinaryMult will generate identical calls during the course of the recursion. To prevent duplication of work, each operation saves its answer in a *cache*, and checks the cache before starting computation. The algorithms also must handle the cases of pointers skipping levels; this is done in lines 11-16 of BinaryAdd and lines 12-15 of BinaryMult , which considers a skipped level to be a matrix whose diagonal pointers are to the next level.

Another operation that we will utilize is determining which rows of a matrix are empty. In terms of a next-state function, this determines which states do not have a next state. This will be used to determine the states that disable a given event. The algorithm to perform this operation is shown in Figure 6. Like the previous algorithms, it is recursive and terminates if the appropriate terminal condition holds or if the answer has already been computed. The terminal conditions are for inputs $\mathbf{I}$, which does not contain any empty rows, and $\mathbf{0}$, which contains all empty rows. Note that the matrix returned by EmptyRows contains non-zero elements only along the diagonal; this allows us to select rows or columns of a matrix using BinaryMult .

Once the proper next-state functions have been determined, generation of the reachability set can proceed using one of the Kronecker-based implicit techniques [14, 15, 27], modified to use matrix diagrams. If our model contains only timed events with equal priority, the next-state functions computed using BuildNext can be used without modi-

```

BinaryMult(Md A, Md B)
1: if (A = zero)  $\vee$  (B = zero) then
2:   return zero;
3: else if (A = one) then
4:   return B;
5: else if (B = one) then
6:   return A;
7: else if MultCache contains(A, B, C) then
8:   return C;
9: end if
10:  $kA \leftarrow \text{Level}(A)$ ;  $kB \leftarrow \text{Level}(B)$ ;
11:  $k \leftarrow \text{MAX}(kA, kB)$ ;
12: C  $\leftarrow$  New level- $k$  node;
13: if  $kA > kB$  then
14:    $\forall i, j, C[i, j] \leftarrow \text{BinaryMult}(A[i, j], B)$ ;
15: else if  $kB > kA$  then
16:    $\forall i, j, C[i, j] \leftarrow \text{BinaryMult}(A, B[i, j])$ ;
17: else
18:   for each  $i, j, l \in S_k$  do
19:     T  $\leftarrow \text{BinaryMult}(A[i, l], B[l, j])$ ;
20:     C[i, j]  $\leftarrow \text{BinaryAdd}(C[i, j], T)$ ;
21:   end for
22: end if
23: Reduce C;
24: Insert MultCache entry (A, B, C);
25: return C;

```

Figure 5. Matrix diagram multiplication

fication. We will briefly outline how to modify the efficient saturation algorithm [15] to use matrix diagrams; details are omitted due to lack of space.

The main procedure of the saturation algorithm is the recursive procedure RecFire, which takes as parameters an event e and a level- k MDD node. From e and k , the appropriate Kronecker matrix is used to manipulate the MDD node. The matrix diagram version instead uses as parameters a matrix diagram node N and an MDD node. The matrix structure of the matrix diagram node is used to manipulate the MDD node. In the Kronecker version, the recursive call to RecFire passes the same event and decrements the level; in the matrix diagram version, the recursive call passes the appropriate downward pointer of the matrix diagram node. For added efficiency, the matrix diagram version can combine matrix diagrams (using BinaryAdd) based on the “first” submodel affected by events.

4. Handling priorities

We are now ready to handle high-level models whose events are allowed to have *priorities*. Specifically, we allow the model to specify, for pairs of events b and c , that event b always has priority over event c . Thus, whenever event b is

```

EmptyRows(Md A)
1: if (A = zero) then
2:   return one;
3: else if (A = one) then
4:   return zero;
5: else if RowsCache contains(A, C) then
6:   return C;
7: end if
8:  $k \leftarrow \text{Level}(A)$ ;
9: C  $\leftarrow$  New level- $k$  node;
10: for each  $i \in S_k$  do
11:   C[i, i]  $\leftarrow$  one;
12:   for each  $j \in S_k$  do
13:     T  $\leftarrow \text{EmptyRows}(A[i, j])$ ;
14:     C[i, j]  $\leftarrow \text{BinaryMult}(C[i, i], T)$ ;
15:   end for
16: end for
17: Reduce C;
18: Insert RowsCache entry (A, C);
19: return C;

```

Figure 6. Determining empty rows of a matrix diagram

enabled, event c effectively becomes disabled. In addition to providing an increase in modeling power, priorities are necessary to handle *immediate* events (events which occur in zero time), since immediate events always have priority over timed events.

To construct matrix diagrams for the next-state functions for each event when events have a priority structure, we first build the next-state functions in isolation, ignoring the priorities, and then modify the functions appropriately. If event b has priority over event c , we must modify function $\mathcal{N}^c$ so that any state s that enables event b has no next-states: $\mathcal{N}^c(s) = \emptyset$. This can be done by manipulating the matrix diagrams N^b and N^c as follows.

1. Determine the states that do not enable event b by computing $\text{EmptyRows}(N^b)$.
2. Restrict event c to these states only, using BinaryMult.

Thus, $\text{BinaryMult}(\text{EmptyRows}(N^b), N^c)$ constructs the proper next-state function for event c , taking into account that event c is disabled whenever event b is enabled.

Complex priority structures can also be handled with the same technique, if care is taken to perform the steps in the correct order. For instance, suppose event a has priority over event b , and event b has priority over event c , but a does not have priority over event c . Then b is disabled whenever a is enabled, and c is disabled whenever b is enabled (and a is disabled). The above computation will correctly determine the next-state function for c , provided N^b has already

```

BuildNextPriority(event  $e$ )
1: if we have computed  $N^e$  then
2:   return;
3: else if we are already computing  $N^e$  then
4:   Error, cyclic dependency;
5: end if
6:  $N^e \leftarrow \text{BuildNext}(\mathcal{N}_K^e, \dots, \mathcal{N}_1^e)$ ;
7:  $E \leftarrow \text{one}$ ;      • States enabling  $e$ , initially all
8: for each event  $e'$  with higher priority than  $e$  do
9:    $\text{BuildNextPriority}(e')$ ;
10:   $E \leftarrow \text{BinaryMult}(E, \text{EmptyRows}(N^{e'}))$ ;
11: end for
12:  $N^e \leftarrow \text{BinaryMult}(E, N^e)$ ;

BuildNextPriorityClass
1:  $E \leftarrow \text{one}$ ;
2:  $F \leftarrow \text{one}$ ;
3: for each priority  $p$ , from high to low do
4:   for each event  $e$  with priority  $p$  do
5:      $N^e \leftarrow \text{BuildNext}(\mathcal{N}_K^e, \dots, \mathcal{N}_1^e)$ ;
6:      $N^e \leftarrow \text{BinaryMult}(F, N^e)$ ;
7:      $E \leftarrow \text{BinaryMult}(E, \text{EmptyRows}(N^e))$ ;
8:   end for
9:    $F \leftarrow E$ ;
10: end for

```

Figure 7. Constructing next-state functions with event priorities

been adjusted. Thus, adjustment of the next-state functions to incorporate priority should proceed from high priority events to low priority events.

Given any acyclic priority structure [13], the next-state function for each event can be determined using the algorithm `BuildNextPriority` in Figure 7. Priority cycles are not allowed (e.g., if event b has priority over c , then event c cannot have priority over b). Existence of a cycle will cause `BuildNextPriority( $e$ )` to eventually call itself with the same parameter, which can be detected. If a priority class structure is used instead, in which each event is assigned an integer priority [1, 11], then the next-state function for each event can be determined using the simpler algorithm `BuildNextPriorityClass`, also shown in Figure 7.

An example illustrating the construction of the next-state functions for events a, b and c , where a has priority over b and b has priority over c , is shown in Figure 8. Event a is enabled in states $(1, \bullet, 0)$ and $(2, \bullet, 0)$. To determine the next-state function for b , we compute $E^a = \text{EmptyRows}(N^a)$ and multiply it by N^b . Note that, before adjustment (matrix diagram N^b in the figure), event b is enabled in states $(2, 2, \bullet)$, but after adjusting for priorities, b is only enabled in state $(2, 2, 1)$. Event c only depends on submodel 2, and is enabled in states $(\bullet, 0, \bullet)$ and $(\bullet, 2, \bullet)$. To adjust its next-

state function to include the priority relationship with b , we compute $E^b = \text{EmptyRows}(E^a \cdot N^b)$ and multiply E^b by N^c . After adjustment, event c is disabled in state $(2, 2, 1)$.

5. Eliminating vanishing states

The technique described in the previous section can be applied to high-level models with event priorities, including GSPNs and other formalisms with immediate events. Reachability set generation will correctly produce the set of reachable states. However, the reachability set $\mathcal{S}$ can be partitioned into the set of *vanishing* states $\mathcal{V}$, in which an immediate event is enabled, and *tangible* states $\mathcal{T}$, in which no immediate events are enabled. For many types of analysis, especially generation and solution of an underlying Markov chain, only the tangible states are of interest, and the vanishing states can be eliminated [3, 4, 12]. As with traditional reachability set generation algorithms, elimination of vanishing states can be done either *during* generation (sometimes called “on-the-fly”) or *after* generation. That is, we can either construct the set $\mathcal{T}$ directly, or we can construct the entire reachability set $\mathcal{S}$, partition it into $\mathcal{T}$ and $\mathcal{V}$, and keep only $\mathcal{T}$.

5.1. Elimination during generation

Elimination of vanishing states during generation must maintain only the set $\mathcal{T}$ and ensure that only the tangible reachable states are added to the set $\mathcal{T}$. This is done by using a modified next-state function $\mathcal{N}'$ that, given a tangible state, returns the set of tangible states that can be reached from the given state when a single timed event occurs, possibly via a sequence of vanishing states. If a tangible state reaches a vanishing state via a timed event, then the sequences of vanishing states are collapsed until another tangible state is reached. If the initial state is tangible, then generating the reachability set using $\mathcal{N}'$ will produce exactly the set $\mathcal{T}$. A vanishing initial state can be handled by determining all the tangible states that can be reached from the initial state via immediate events only, and using those as the initial states.

If the next-state function is represented as a matrix N , then we can classify the rows and columns as either tangible or vanishing, and obtain the block structure

$$N = \begin{bmatrix} N_{\mathcal{T}} \\ N_{\mathcal{V}} \end{bmatrix} = \begin{bmatrix} N_{\mathcal{T}\mathcal{T}} & N_{\mathcal{T}\mathcal{V}} \\ N_{\mathcal{V}\mathcal{T}} & N_{\mathcal{V}\mathcal{V}} \end{bmatrix}$$

where $N_{\mathcal{T}\mathcal{T}}$ describes transitions from tangible states to tangible states, $N_{\mathcal{T}\mathcal{V}}$ describes transitions from tangible states to vanishing states, and so on. The matrix representation N' is exactly

$$N' = N_{\mathcal{T}\mathcal{T}} + N_{\mathcal{T}\mathcal{V}} \cdot N_{\mathcal{V}\mathcal{V}}^* \cdot N_{\mathcal{V}\mathcal{T}} \quad (3)$$

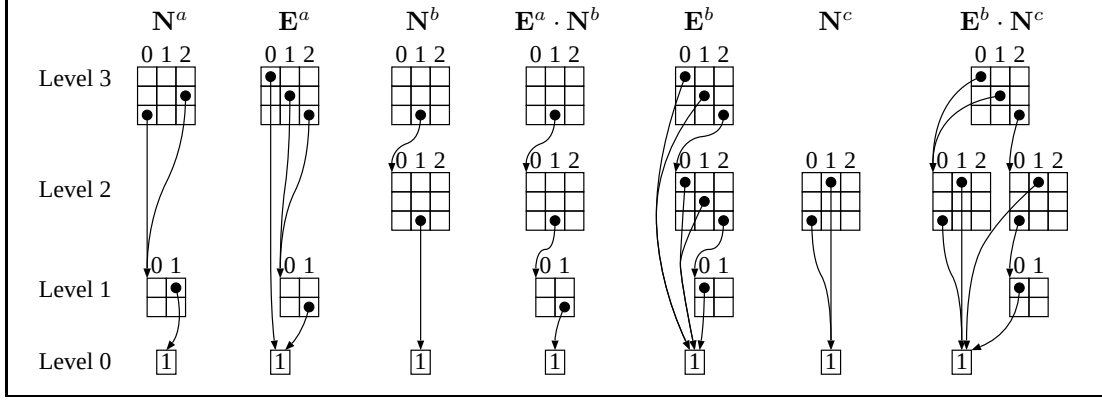


Figure 8. Example next-state construction for events a, b, c

where “*” denotes transitive and reflexive closure. Thus, to get from a tangible state to another tangible state via a single timed event is possible directly, by a transition from a tangible state to a tangible state, or via vanishing states, by a transition from a tangible state to a vanishing state, followed by zero or more transitions from a vanishing state to another vanishing state, followed by a transition from a vanishing state to a tangible state.

To construct the matrix diagram N' , we use Equation 3. Given the next-state function for each event, modified to account for priorities (as described earlier), we can compute N_T by summing the next-state functions for the timed events, and N_V by summing the next-state functions for the immediate events, using BinaryAdd. The potential tangible states can be found by computing $P_T = \text{EmptyRows}(N_V)$, and the potential vanishing states can be found by computing $P_V = \text{EmptyRows}(P_T)$. The tangible (vanishing) columns of a matrix can be selected by multiplying the matrix by P_T (P_V). For instance, we compute $N_{TT} = \text{BinaryMult}(N_T, P_T)$ and $N_{TV} = \text{BinaryMult}(N_T, P_V)$. To compute the transitive and reflexive closure of N_{VV} , we compute $N_{VV}^* = (I + N_{VV})^\infty$ by iterative squaring: we initialize N_{VV}^* to $I + N_{VV}$, and repeatedly compute $N_{VV}^* \leftarrow (N_{VV}^*)^2$ using BinaryMult until convergence is achieved.

5.2. Elimination after generation

Elimination of the vanishing states after the reachability set has been generated is fairly straightforward. The next-state functions are constructed as described in Section 4, with the immediate events having priority over the timed events, and are summed to obtain the overall next-state function N . Using the saturation algorithm, we generate the set of reachable states $S = T \cup V$. The set of tangible, reachable states T can be determined using a simple query, as described in [27]. First, we determine the set of poten-

tial states that are tangible, by determining the states that do not enable any immediate events. This is done by computing $P_T = \text{EmptyRows}(N_V)$, as described earlier. The non-empty rows of P_T correspond to the potential tangible states. We then compute the intersection of the reachability set with the set of potential tangible states, which gives us the set of tangible, reachable states.

6. Experimental results

Prototypes of both “elimination after generation” and “elimination during generation” are implemented in the SMART tool [16]. All results are obtained from 933 MHz Pentium III workstations with 512Mb of RAM, running Linux. No reported results made use of virtual memory.

We examine several models taken from the literature, each parameterized by an integer N . The dining philosopher model, taken from [27, 30], is composed of N subnets, one for each philosopher. Our model differs from the models in [27, 30]: we use immediate transitions instead of timed transitions for the action of picking up a fork. The dining philosophers net is broken into subnets so that each subnet consists of two adjacent philosophers. We also study a kanban network and a flexible manufacturing system (FMS), taken from [17] and [20]. These models are fixed in size but have N tokens initially in certain places, corresponding to parts or jobs that circulate in the system. We use the “immediate” version of the kanban model, which has immediate synchronizing transitions. The FMS model is modified from its original presentation [20] so that all arc cardinalities are one. Finally, we consider a multiserver polling system, described in [2], using the $1 \times Q$ server utilization policy. The system consists of multiple queues, and servers that move from queue to queue. The Polling model is composed of a subnet for each queue, and contains multiple tokens corresponding to the numbers of

Model	$ \mathcal{T} $	Elimination during generation					Elimination after generation				
		Generating $\mathcal{N}'$			Generating $\mathcal{T}$		Generating $\mathcal{N}$			Generating $\mathcal{T}$	
		CPU (sec)	Memory (Kb) Final	Memory (Kb) Peak	CPU (sec)	Memory (Kb) Peak (Kb)	CPU (sec)	Memory (Kb) Final	Memory (Kb) Peak	CPU (sec)	Memory (Kb) Peak (Kb)
Phils 16	4.87×10^6	0.2	74	174	0.8	1,051	0.04	43	133	1.3	2,382
Phils 30	3.46×10^{12}	1.2	150	631	19	16,637	0.08	85	262	10	16,603
Phils 60	1.20×10^{25}	9.5	314	2,508	389	166,520	0.21	176	539	69	91,972
Phils 90	4.15×10^{37}	—	—	—	—	—	0.39	266	815	204	228,561
Kanban 8	4.23×10^7	0.3	42	188	1.1	878	0.00	23	40	0.5	511
Kanban 30	2.36×10^{12}	96	147	22,275	2,157	193,920	0.02	80	144	67	35,594
Kanban 40	2.86×10^{13}	—	—	—	—	—	0.03	106	191	280	99,346
Kanban 50	2.01×10^{14}	—	—	—	—	—	0.04	132	238	979	224,295
FMS 8	4.59×10^6	2	23	1,758	0.1	153	0.01	15	33	0.2	228
FMS 20	8.83×10^9	356	51	63,036	1.5	1,376	0.01	33	74	2.5	2,194
FMS 40	4.97×10^{12}	—	—	—	—	—	0.03	64	144	29	14,311
FMS 80	3.71×10^{15}	—	—	—	—	—	0.05	124	282	447	102,070
Poll 5	5.91×10^6	0.1	18	62	0.2	220	0.01	12	31	0.4	390
Poll 10	9.34×10^{16}	9.4	68	1,784	4.6	4,677	0.03	44	111	13	8,795
Poll 15	2.28×10^{28}	297	147	15,284	36	29,288	0.07	96	240	113	56,180
Poll 20	3.20×10^{40}	—	—	—	—	—	0.14	167	418	540	214,350

Table 1. Time and space requirements to construct the tangible reachability set $\mathcal{T}$

jobs and servers in the system. Our model uses N queues, jobs, and servers; thus, both the model size and initial numbers of tokens increase with N . For the Kanban, FMS, and Polling models, we use the finest possible decomposition: one place per subnet.

Table 1 compares the costs, in terms of computational and storage requirements, of constructing the tangible reachability set $\mathcal{T}$ for elimination during generation versus elimination after generation. When vanishing states are eliminated during generation, we construct the next-state function $\mathcal{N}'$ using Equation 3 and then generate $\mathcal{T}$ directly. Conversely, eliminating vanishing states after generation requires construction of the next-state function $\mathcal{N}$, generation of the reachable states $\mathcal{S}$, and determination of $\mathcal{T}$ using a query on $\mathcal{S}$. For both techniques, we show the costs of constructing the appropriate next-state function, and the costs of constructing the set $\mathcal{T}$. Since MDDs and matrix diagrams may expand and contract during manipulation, both the peak and final memory usage is reported. For next-state function construction, the memory usage reported is the total memory required for the matrix diagram. For the generation of $\mathcal{T}$, the memory usage reported is the total memory required for MDD nodes. Since the resulting MDD encoding $\mathcal{T}$ is the same for both techniques, we do not report the final memory usage for $\mathcal{T}$.

As we expect, constructing the next-state function $\mathcal{N}'$ is more expensive than constructing $\mathcal{N}$, since the construction of $\mathcal{N}'$ requires the construction of $\mathcal{N}$ and then application of Equation 3. However, we see that, as the model size

increases, the space and time requirements to compute $\mathcal{N}'$ grow much faster than those to compute $\mathcal{N}$. This rapid increase is due to the transitive closure computation in Equation 3. Iterative squaring is not the best way to compute transitive closure; we are currently investigating more efficient implicit techniques [24]. The dashes in the table correspond to cases in which the storage requirements to construct $\mathcal{N}'$ exceeded available memory.

While our intuition may suggest that elimination during generation will require less memory than elimination after generation, this is not always the case. This is due to the property of MDDs that the computational and storage complexities for manipulation depend on the number of *nodes* in the MDD, not the number of states encoded by the MDD. Removal of states from an MDD may actually increase its storage requirements! For example, computational and storage requirements for reachability set generation for the dining philosophers model are significantly lower when all transitions are timed [15]. As both the computational and storage complexities depend on the number of nodes in the MDD, there is a correlation between the peak memory and CPU time for generation of $\mathcal{T}$. That is, for all reported results, whichever technique requires less storage space to generate $\mathcal{T}$ also requires less time. However, there is no clear trend as to when the generation of $\mathcal{T}$ is more efficient if elimination during generation is used versus elimination after generation. When a comparison is possible, we see that elimination during generation is more efficient for the FMS and Polling models, and elimination

after generation is more efficient for the Kanban model. For the dining philosophers model, elimination during generation is more efficient for small models, while elimination after generation is more efficient for large models. All things considered, the cost of constructing $\mathcal{N}'$ tends to make elimination during generation more expensive and less practical.

Table 1 also illustrates the power of implicit techniques. The first row for each model in the table corresponds to the largest tangible reachability set that could be generated using explicit techniques. Clearly, implicit techniques allow for the study of much larger systems than would otherwise be possible with explicit techniques.

7. Conclusion

Implicit techniques for reachability set generation are quite promising, as they are often able to handle extremely large sets, but are usually limited to handling restricted classes of models. We addressed one such limitation of current implicit reachability set generation techniques: their inability to handle events with priorities and immediate events. We presented an alternate representation of the next-state function of a model, based on matrix diagrams. Manipulations of the matrix diagrams allow for construction of next-state functions that can handle models with event priorities and immediate events. We showed how vanishing states can be eliminated either during generation or after generation, when only the tangible states are of interest.

While the costs of state generation can be lower when vanishing states are eliminated during generation, the approach is often impractical due to the extra computation required to construct the adjusted next-state function. This is mainly due to the cost of computing the transitive and reflexive closure of the vanishing-to-vanishing next-state function. One possible direction for further research is the development of improved techniques that reduce this cost.

References

- [1] M. Ajmone Marsan, G. Balbo, G. Chiola, and G. Conte. Generalized stochastic Petri nets revisited: random switches and priorities. In *2nd Int. Workshop on Petri Nets and Performance Models (PNPM'87)*, pages 44–53, Madison, Wisconsin, Aug. 1987. IEEE Comp. Soc. Press.
- [2] M. Ajmone Marsan, S. Donatelli, F. Neri, and A. Scalia. Approximate GSPN analysis of multiserver polling systems. In *Proceedings of the International Conference on Telecommunication, Distribution and Parallelism (TDP'96)*, La Londe Les Maures, France, June 1996.
- [3] G. Balbo, G. Chiola, G. Franceschinis, and G. Molinari Roet. On the efficient construction of the tangible reachability graph of generalized stochastic Petri nets. In *2nd Int. Workshop on Petri Nets and Performance Models (PNPM'87)*, pages 136–145, Madison, Wisconsin, Aug. 1987. IEEE Comp. Soc. Press.
- [4] A. Blakemore. The cost of eliminating vanishing markings from generalized stochastic Petri nets. In *3rd Int. Workshop on Petri Nets and Performance Models (PNPM'89)*, pages 85–92, Kyoto, Japan, Dec. 1989. IEEE Comp. Soc. Press.
- [5] R. E. Bryant. Graph-based algorithms for boolean function manipulation. *IEEE Trans. Comp.*, C-35(8):677–691, Aug. 1986.
- [6] P. Buchholz. Hierarchical structuring of superposed GSPNs. In *7th Int. Workshop on Petri Nets and Performance Models (PNPM'97)*, pages 81–90, Saint Malo, France, June 1997. IEEE Comp. Soc. Press.
- [7] P. Buchholz, G. Ciardo, S. Donatelli, and P. Kemper. Complexity of memory-efficient Kronecker operations with applications to the solution of Markov models. *INFORMS J. Comp.*, 12(3):203–222, SUMMER 2000.
- [8] J. Burch, E. Clarke, D. Long, K. McMillan, and D. Dill. Symbolic model checking for sequential circuit verification. *IEEE Trans. Computer-Aided Design of Integrated Circuits and Systems*, 13(4):401–424, Apr. 1994.
- [9] J. R. Burch, E. M. Clarke, K. L. McMillan, D. L. Dill, and L. J. Hwang. Symbolic model checking: 10^{20} states and beyond. *Information and Computation*, 98(2):142–170, June 1992.
- [10] G. Chiola. Compiling techniques for the analysis of stochastic Petri nets. *Perf. Eval.*, Sept. 1988.
- [11] G. Chiola, M. Ajmone Marsan, G. Balbo, and G. Conte. Generalized stochastic Petri nets: a definition at the net level and its implications. *IEEE Trans. Softw. Eng.*, 19(2):89–107, Feb. 1993.
- [12] G. Ciardo. *Analysis of large stochastic Petri net models*. PhD thesis, Duke University, Durham, NC, 1989.
- [13] G. Ciardo. Discrete-time Markovian stochastic Petri nets. In W. J. Stewart, editor, *Computations with Markov Chains*, pages 339–358. Kluwer, Boston, MA, 1995.
- [14] G. Ciardo, G. Luetzgen, and R. Siminiceanu. Efficient symbolic state-space construction for asynchronous

- systems. In M. Nielsen and D. Simpson, editors, *Application and Theory of Petri Nets 2000 (Proc. 21st Int. Conf. on Applications and Theory of Petri Nets)*, LNCS 1825, pages 103–122, Aarhus, Denmark, June 2000. Springer-Verlag.
- [15] G. Ciardo, G. Luetzgen, and R. Siminiceanu. Saturation: an efficient iteration strategy for symbolic state space generation. In T. Margaria and W. Yi, editors, *Proc. Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, LNCS 2031, pages 328–342, Genova, Italy, Apr. 2001. Springer-Verlag.
- [16] G. Ciardo and A. S. Miner. SMART: Simulation and Markovian Analyzer for Reliability and Timing. In *Proc. IEEE International Computer Performance and Dependability Symposium (IPDS'96)*, page 60, Urbana-Champaign, IL, USA, Sept. 1996. IEEE Comp. Soc. Press.
- [17] G. Ciardo and A. S. Miner. Storage alternatives for large structured state spaces. In R. Marie, B. Plateau, M. Calzarossa, and G. Rubino, editors, *Proc. 9th Int. Conf. on Modelling Techniques and Tools for Computer Performance Evaluation*, LNCS 1245, pages 44–57, Saint Malo, France, June 1997. Springer-Verlag.
- [18] G. Ciardo and A. S. Miner. A data structure for the efficient Kronecker solution of GSPNs. In P. Buchholz, editor, *8th Int. Workshop on Petri Nets and Performance Models (PNPM'99)*, pages 22–31, Zaragoza, Spain, Sept. 1999. IEEE Comp. Soc. Press.
- [19] G. Ciardo and A. S. Miner. Structural approaches for SPN analysis. In *High Performance Computing 2000, Grand Challenges in Computer Simulation*, pages 345–356, Washington, DC, Apr. 2000.
- [20] G. Ciardo and K. S. Trivedi. A decomposition approach for stochastic reward net models. *Perf. Eval.*, 18(1):37–59, 1993.
- [21] T. Kam, T. Villa, R. Brayton, and A. Sangiovanni-Vincentelli. Multi-valued decision diagrams: theory and applications. *Multiple-Valued Logic*, 4(1–2):9–62, 1998.
- [22] P. Kemper. Numerical analysis of superposed GSPNs. *IEEE Trans. Softw. Eng.*, 22(9):615–628, Sept. 1996.
- [23] P. Kemper. Reachability analysis based on structured representations. In J. Billington and W. Reisig, editors, *Application and Theory of Petri Nets 1996 (Proc. 17th Int. Conf. on Applications and Theory of Petri Nets)*, LNCS 1091, pages 269–288, Osaka, Japan, June 1996. Springer-Verlag.
- [24] Y. Matsunaga, P. C. McGeer, and R. K. Brayton. On computing the transitive closure of a state transition relation. In *Proc. 30th International Design Automation Conference*, pages 260–265. ACM Press, 1993.
- [25] A. S. Miner. *Data Structures for the Analysis of Large Structured Markov Models*. PhD thesis, The College of William and Mary, Williamsburg, VA, 2000.
- [26] A. S. Miner. Efficient solution of GSPNs using Canonical Matrix Diagrams. In R. German and B. Haverkort, editors, *9th Int. Workshop on Petri Nets and Performance Models (PNPM'01)*, pages 101–110, Aachen, Germany, Sept. 2001. IEEE Comp. Soc. Press.
- [27] A. S. Miner and G. Ciardo. Efficient reachability set generation and storage using decision diagrams. In H. Kleijn and S. Donatelli, editors, *Application and Theory of Petri Nets 1999 (Proc. 20th Int. Conf. on Applications and Theory of Petri Nets)*, LNCS 1639, pages 6–25, Williamsburg, VA, USA, June 1999. Springer-Verlag.
- [28] A. S. Miner, G. Ciardo, and S. Donatelli. Using the exact state space of a Markov model to compute approximate stationary measures. In *Proc. 2000 ACM SIGMETRICS Conf. on Measurement and Modeling of Computer Systems*, pages 207–216, Santa Clara, CA, June 2000.
- [29] E. Pastor, J. Cortadella, and M. A. Peña. Structural methods to improve the symbolic analysis of Petri nets. In H. Kleijn and S. Donatelli, editors, *Application and Theory of Petri Nets 1999 (Proc. 20th Int. Conf. on Applications and Theory of Petri Nets)*, LNCS 1639, pages 26–45, Williamsburg, VA, USA, June 1999. Springer-Verlag.
- [30] E. Pastor, O. Roig, J. Cortadella, and R. Badia. Petri net analysis using boolean manipulation. In R. Valette, editor, *Application and Theory of Petri Nets 1994 (Proc. 15th Int. Conf. on Applications and Theory of Petri Nets)*, LNCS 815, pages 416–435, Zaragoza, Spain, June 1994. Springer-Verlag.
- [31] B. Plateau. On the stochastic structure of parallelism and synchronisation models for distributed algorithms. In *Proc. 1985 ACM SIGMETRICS Conf. on Measurement and Modeling of Computer Systems*, pages 147–153, Austin, TX, USA, May 1985.
- [32] A. Srinivasan, T. Kam, S. Malik, and R. K. Brayton. Algorithms for discrete function manipulation. In *International Conference on CAD*, pages 92–95. IEEE Comp. Soc. Press, 1990.